

HookSuite — Manual Técnico P1 Frontend

Proyecto: HookSuite

Rol: P1 — Frontend

Stack: React + Vite + Tailwind CSS

Última actualización: Abril 2026

Antes de empezar

Este manual es tu guía completa de trabajo en el proyecto HookSuite. Está diseñado para que puedas trabajar de forma autónoma desde cualquier lugar y en cualquier momento. Cada paso está detallado al máximo para que no tengas que detenerte a investigar cómo hacer algo.

Cuando un paso requiere que te coordines con otro rol, el manual te lo indica claramente con este bloque:

👉 **CONTACTO CON P2 Backend** — Lo que necesitas pedirle y cómo integrarlo.

Cuando un paso es complejo y la IA puede ayudarte, encontrarás este bloque con el prompt listo para copiar y pegar:

🤖 **PROMPT DE IA** — Copia este prompt en Claude Code o Claude.ai y obtendrás exactamente lo que necesitas.

Herramientas que necesitas tener instaladas

Antes de empezar con el paso 1, verifica que tienes todo esto instalado en tu máquina:

- **Node.js 20 o superior** — Descarga desde nodejs.org. Verifica con `node --version` en la terminal.
- **npm** — Viene con Node.js. Verifica con `npm --version`.
- **Git** — Verifica con `git --version`.
- **Visual Studio Code** — O el editor que prefieras, pero este manual asume VS Code.
- **Claude Code** — El cliente de terminal de Anthropic. Úsalo para los prompts de IA de este manual.

BLOQUE 01 — Estructura base del proyecto

Paso 1 — Clonar el repositorio y crear tu rama de trabajo

1.1 Abre la terminal en tu máquina y navega a la carpeta donde quieres trabajar.

1.2 Clona el repositorio de HookSuite:

```
git clone https://github.com/[URL-DEL-REPO]/hooksuite.git
cd hooksuite
```

1.3 Crea tu rama de trabajo. Nunca trabajarás directamente en `main`:

```
git checkout -b feature/frontend
```

1.4 Verifica que estás en la rama correcta:

```
git branch
```

Debe aparecer `* feature/frontend`.

Paso 2 — Inicializar el proyecto React con Vite

2.1 Dentro de la carpeta del repositorio, navega a la carpeta del frontend:

```
cd frontend
```

2.2 Inicializa el proyecto React con Vite:

```
npm create vite@latest . -- --template react
```

Cuando pregunte si quieres continuar en la carpeta actual, responde `y`.

2.3 Instala las dependencias base:

```
npm install
```

2.4 Verifica que el proyecto arranca correctamente:

```
npm run dev
```

Abre el navegador en `http://localhost:5173`. Debes ver la pantalla de bienvenida de Vite + React. Si la ves, el setup es correcto.

2.5 Para el servidor con `Ctrl + C`.

2.6 Instala las dependencias que vas a usar a lo largo del proyecto:

```
npm install react-router-dom tailwindcss @tailwindcss/vite axios @tanstack/react-
```

```
virtual diff codemirror @codemirror/lang-json @codemirror/lang-html
```

2.7 Configura Tailwind CSS. Abre el archivo `vite.config.js` y reemplaza su contenido con:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
    react(),
    tailwindcss(),
  ],
  resolve: {
    alias: {
      '@': '/src',
    },
  },
})
```

2.8 Abre el archivo `src/index.css` y reemplaza todo su contenido con:

```
@import "tailwindcss";
```

 **PROMPT DE IA** — Si tienes problemas con la configuración de Tailwind, copia este prompt en Claude Code:

```
Estoy configurando Tailwind CSS v4 con Vite y React. Mi vite.config.js usa
@tailwindcss/vite como plugin y mi index.css solo tiene @import
"tailwindcss". El proyecto no aplica los estilos de Tailwind. Revisa mi
configuración y dime qué está mal.
```

Paso 3 — Crear la estructura de carpetas

3.1 Dentro de la carpeta `src`, crea la siguiente estructura de carpetas. Puedes hacerlo desde la terminal:

```
mkdir -p src/components/ui
mkdir -p src/components/layout
mkdir -p src/pages/proxy
mkdir -p src/pages/repeater
mkdir -p src/pages/intruder
mkdir -p src/pages/utilities
mkdir -p src/pages/vulnerabilities
mkdir -p src/pages/network
```

```
mkdir -p src/hooks
mkdir -p src/services
mkdir -p src/utils
```

3.2 Elimina los archivos que Vite genera por defecto y que no necesitas:

```
rm src/App.css
rm src/assets/react.svg
```

3.3 La estructura final de `src` debe quedar así:

```
src/
├── components/
│   ├── ui/           ← Botones, inputs, badges, modales
│   └── layout/       ← Sidebar, header, layout principal
├── pages/
│   ├── proxy/        ← Módulo interceptor
│   ├── repeater/     ← Módulo repeater
│   ├── intruder/     ← Módulo intruder
│   ├── utilities/    ← Encoder, hash, regex, payloads
│   ├── vulnerabilities/ ← Panel de alertas de IA
│   └── network/      ← Panel de red en tiempo real
├── hooks/            ← Custom hooks de React
├── services/         ← Comunicación con el backend y mocks
├── utils/            ← Funciones auxiliares
├── App.jsx
├── main.jsx
└── index.css
```

Paso 4 — Crear los componentes de sistema de diseño base

Estos componentes son los ladrillos de toda la interfaz. Créalos bien ahora y ahorrarás mucho tiempo después.

4.1 Crea el componente `Button` en `src/components/ui/Button.jsx`:

```
export function Button({ children, variant = 'primary', size = 'md', onClick,
disabled, className = '' }) {
  const base = 'inline-flex items-center justify-center font-medium rounded-lg
transition-colors focus:outline-none focus:ring-2 focus:ring-offset-2
disabled:opacity-50 disabled:cursor-not-allowed'
  const variants = {
    primary: 'bg-blue-600 text-white hover:bg-blue-700 focus:ring-blue-500',
    secondary: 'bg-gray-100 text-gray-900 hover:bg-gray-200 focus:ring-gray-500',
    danger: 'bg-red-600 text-white hover:bg-red-700 focus:ring-red-500',
    ghost: 'text-gray-600 hover:bg-gray-100 focus:ring-gray-500',
```

```

    }
    const sizes = {
      sm: 'px-3 py-1.5 text-sm',
      md: 'px-4 py-2 text-sm',
      lg: 'px-6 py-3 text-base',
    }
    return (
      <button
        onClick={onClick}
        disabled={disabled}
        className={` ${base} ${variants[variant]} ${sizes[size]} ${className}`}
      >
        {children}
      </button>
    )
  }
}

```

4.2 Crea el componente `Badge` en `src/components/ui/Badge.jsx`:

```

export function Badge({ children, variant = 'default' }) {
  const variants = {
    default: 'bg-gray-100 text-gray-800',
    get: 'bg-blue-100 text-blue-800',
    post: 'bg-green-100 text-green-800',
    put: 'bg-yellow-100 text-yellow-800',
    delete: 'bg-red-100 text-red-800',
    critical: 'bg-red-100 text-red-800',
    high: 'bg-orange-100 text-orange-800',
    medium: 'bg-yellow-100 text-yellow-800',
    low: 'bg-gray-100 text-gray-600',
    success: 'bg-green-100 text-green-800',
    warning: 'bg-yellow-100 text-yellow-800',
    danger: 'bg-red-100 text-red-800',
  }
  return (
    <span className={`inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-medium ${variants[variant]} || variants.default`} >
      {children}
    </span>
  )
}

```

4.3 Crea el componente `Spinner` en `src/components/ui/Spinner.jsx`:

```

export function Spinner({ size = 'md' }) {
  const sizes = { sm: 'h-4 w-4', md: 'h-6 w-6', lg: 'h-8 w-8' }
  return (
    <div className={`animate-spin rounded-full border-2 border-gray-300 border-t-blue-600 ${sizes[size]} ` />

```

```
)
}
```

4.4 Crea el componente `Panel` en `src/components/ui/Panel.jsx`:

```
export function Panel({ children, className = '' }) {
  return (
    <div className={`bg-white border border-gray-200 rounded-lg ${className}`}>
      {children}
    </div>
  )
}
```

4.5 Exporta todos los componentes desde un índice en `src/components/ui/index.js`:

```
export { Button } from './Button'
export { Badge } from './Badge'
export { Spinner } from './Spinner'
export { Panel } from './Panel'
```

 **PROMPT DE IA** — Para crear componentes adicionales que necesites, usa este prompt en Claude Code:

Estoy construyendo HookSuite, una herramienta de pentesting web. Necesito un componente React con Tailwind CSS para [describe el componente]. Debe seguir el mismo estilo que mis otros componentes: fondo blanco, bordes grises suaves, esquinas redondeadas. Dame el componente completo listo para usar.

Paso 5 — Crear el layout principal con React Router

5.1 Reemplaza el contenido de `src/App.jsx` con:

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom'
import { Layout } from '@components/layout/Layout'
import { ProxyPage } from '@pages/proxy/ProxyPage'
import { RepeaterPage } from '@pages/repeater/RepeaterPage'
import { IntruderPage } from '@pages/intruder/IntruderPage'
import { UtilitiesPage } from '@pages/utilities/UtilitiesPage'
import { VulnerabilitiesPage } from '@pages/vulnerabilities/VulnerabilitiesPage'
import { NetworkPage } from '@pages/network/NetworkPage'

export default function App() {
  return (
    <BrowserRouter>
```

```

    <Routes>
      <Route path="/" element={<Layout />} />
      <Route index element={<Navigate to="/proxy" replace />} />
      <Route path="proxy" element={<ProxyPage />} />
      <Route path="repeater" element={<RepeaterPage />} />
      <Route path="intruder" element={<IntruderPage />} />
      <Route path="utilities" element={<UtilitiesPage />} />
      <Route path="vulnerabilities" element={<VulnerabilitiesPage />} />
      <Route path="network" element={<NetworkPage />} />
    </Route>
  </Routes>
</BrowserRouter>
)
}

```

5.2 Crea el componente `Layout` en `src/components/layout/Layout.jsx`:

```

import { Outlet, NavLink } from 'react-router-dom'

const navItems = [
  { path: '/proxy', label: 'Proxy', icon: '🔗' },
  { path: '/repeater', label: 'Repeater', icon: '🏠' },
  { path: '/intruder', label: 'Intruder', icon: '⚡' },
  { path: '/utilities', label: 'Utilidades', icon: '🔧' },
  { path: '/vulnerabilities', label: 'Vulnerabilidades', icon: '⚠️' },
  { path: '/network', label: 'Red', icon: '🌐' },
]

export function Layout() {
  return (
    <div className="flex h-screen bg-gray-50">
      <aside className="w-56 bg-gray-900 flex flex-col">
        <div className="px-6 py-4 border-b border-gray-700">
          <h1 className="text-white font-bold text-lg">HookSuite</h1>
          <p className="text-gray-400 text-xs mt-0.5">Web Security Toolkit</p>
        </div>
        <nav className="flex-1 px-3 py-4 space-y-1">
          {navItems.map(item => (
            <NavLink
              key={item.path}
              to={item.path}
              className={({ isActive }) =>
                `flex items-center gap-3 px-3 py-2 rounded-lg text-sm transition-
colors ${
              isActive
                ? 'bg-blue-600 text-white'
                : 'text-gray-300 hover:bg-gray-800 hover:text-white'
            }`
            >
              <span>{item.icon}</span>
              {item.label}
            </div>
          )
        </nav>
      </aside>
      <div>
        <Outlet />
      </div>
    </div>
  )
}

```

```

        </NavLink>
      )})
    </nav>
    <div className="px-6 py-4 border-t border-gray-700">
      <p className="text-gray-500 text-xs">HookSuite v1.0</p>
    </div>
  </aside>
  <main className="flex-1 overflow-auto">
    <Outlet />
  </main>
</div>
)
}

```

5.3 Crea páginas vacías temporales para que el router no dé errores. Créalas todas con el mismo contenido inicial. Por ejemplo `src/pages/proxy/ProxyPage.jsx`:

```

export function ProxyPage() {
  return (
    <div className="p-6">
      <h2 className="text-xl font-semibold text-gray-900">Proxy Interceptor</h2>
      <p className="text-gray-500 mt-1">En construcción</p>
    </div>
  )
}

```

Repite esto para `RepeaterPage`, `IntruderPage`, `UtilitiesPage`, `VulnerabilitiesPage` y `NetworkPage` cambiando el título de cada una.

5.4 Arranca el proyecto y verifica que el layout funciona:

```
npm run dev
```

Debes ver el sidebar oscuro con HookSuite a la izquierda y el área de contenido a la derecha. La navegación entre secciones debe funcionar.

Paso 6 — Crear los mocks de datos

Los mocks son datos ficticios que simulan lo que enviará P2 Backend por WebSocket. Te permiten desarrollar toda la interfaz sin esperar a que el backend esté listo.

6.1 Crea el archivo `src/services/mockData.js`:

```

export const mockRequests = [
  {
    id: '1',

```

```
method: 'POST',
url: 'http://dvwa.local/login.php',
status: 200,
size: 4823,
time: 142,
timestamp: new Date().toISOString(),
requestHeaders: { 'Content-Type': 'application/x-www-form-urlencoded',
'Cookie': 'PHPSESSID=abc123' },
requestBody: 'username=admin&password=password&Login=Login',
responseHeaders: { 'Content-Type': 'text/html; charset=UTF-8', 'Server':
'Apache/2.4.51' },
responseBody: '<html><body>Welcome to DVWA</body></html>',
suspicious: false,
vulnerable: false,
},
{
id: '2',
method: 'GET',
url: 'http://dvwa.local/vulnerabilities/sqli/?id=1&Submit=Submit',
status: 200,
size: 5102,
time: 89,
timestamp: new Date().toISOString(),
requestHeaders: { 'Cookie': 'PHPSESSID=abc123; security=low' },
requestBody: null,
responseHeaders: { 'Content-Type': 'text/html; charset=UTF-8' },
responseBody: '<html><body>ID: 1<br>First name: admin<br>Surname: admin</body>
</html>',
suspicious: true,
vulnerable: false,
},
{
id: '3',
method: 'GET',
url: "http://dvwa.local/vulnerabilities/sqli/?id=' OR '1'='1&Submit=Submit",
status: 200,
size: 8934,
time: 312,
timestamp: new Date().toISOString(),
requestHeaders: { 'Cookie': 'PHPSESSID=abc123; security=low' },
requestBody: null,
responseHeaders: { 'Content-Type': 'text/html; charset=UTF-8' },
responseBody: '<html><body>ID: admin, First name: admin...</body></html>',
suspicious: true,
vulnerable: true,
},
]

export const mockVulnerabilities = [
{
id: 'vuln-1',
tipo: 'SQL Injection',
severidad: 'critical',
titulo: 'SQL Injection en parámetro id',
```

```

    descripcion: 'El parámetro id en /vulnerabilities/sqli/ no está sanitizado
correctamente. Es posible extraer toda la base de datos.',
    url: 'http://dvwa.local/vulnerabilities/sqli/',
    payload: "' OR '1'='1'",
    recomendacion: 'Usar consultas preparadas con parámetros enlazados. Nunca
concatenar input del usuario en consultas SQL.',
    timestamp: new Date().toISOString(),
  },
]

export const mockConsoleErrors = [
  {
    level: 'error',
    message: 'MySQL error: You have an error in your SQL syntax near \'1\`\'',
    url: 'http://dvwa.local/vulnerabilities/sqli/index.php',
    line: 42,
    timestamp: new Date().toISOString(),
  },
]

```

7.2 Crea el servicio que decide si usar mocks o datos reales en `src/services/api.js`:

```

const USE MOCKS = true // Cambiar a false cuando P2 Backend esté listo

const API_BASE = import.meta.env.VITE_API_URL || 'http://localhost:8000'

export const config = { USE MOCKS, API_BASE }

```

 **PROMPT DE IA** — Si necesitas añadir más datos de mock para un caso específico:

Estoy desarrollando HookSuite, una herramienta de pentesting web. Necesito datos de mock en JavaScript para simular [describe qué necesitas: peticiones HTTP interceptadas / vulnerabilidades detectadas / errores de consola]. Los datos deben tener este formato: [pega el formato de mockData.js]. Genera 5 ejemplos realistas.

Paso 7 — Primer commit

7.1 Añade todos los archivos al staging:

```
git add .
```

7.2 Haz el primer commit siguiendo la convención del proyecto:

```
git commit -m "feat(frontend): estructura base React + Vite + Tailwind + layout principal"
```

7.3 Sube tu rama al repositorio remoto:

```
git push origin feature/frontend
```

Con esto el Bloque 01 está completo. Tienes el proyecto inicializado, la estructura de carpetas, el sistema de diseño base, el layout con navegación y los mocks de datos.

BLOQUE 02 — Módulo Proxy / Interceptor

Paso 8 — Crear el hook de WebSocket

Este hook gestiona la conexión con el backend de P2. Lo construyes ahora aunque P2 no esté listo: usa los mocks hasta que P2 tenga el WebSocket funcionando.

📌 **CONTACTO CON P2 Backend** — Antes de completar este paso necesitas pedirle a P2 dos cosas:

1. La URL del WebSocket cuando esté disponible (formato: `ws://localhost:8000/ws/{token}`)
2. El formato exacto del JSON que emite por WebSocket para una petición interceptada

Mientras tanto trabaja con los mocks. Cuando P2 te confirme la URL y el formato, actualiza la constante `WS_URL` en el hook.

8.1 Crea el hook `src/hooks/useWebSocket.js`:

```
import { useEffect, useRef, useState, useCallback } from 'react'
import { config } from '@services/api'
import { mockRequests } from '@services/mockData'

const WS_URL = `ws://${config.API_BASE.replace('http://', '').replace('https://', '')}/ws`

export function useWebSocket(sessionToken) {
  const [requests, setRequests] = useState(config.USE MOCKS ? mockRequests : [])
  const [connected, setConnected] = useState(config.USE MOCKS)
  const wsRef = useRef(null)

  useEffect(() => {
    if (config.USE MOCKS) return

    const ws = new WebSocket(`${WS_URL}/${sessionToken}`)
    wsRef.current = ws

    ws.onopen = () => setConnected(true)
```

```

ws.onclose = () => setConnected(false)

ws.onmessage = (event) => {
  const data = JSON.parse(event.data)
  if (data.type === 'request_intercepted') {
    setRequests(prev => [data.payload, ...prev])
  }
}

return () => ws.close()
}, [sessionToken])

const clearRequests = useCallback(() => setRequests([], []))

return { requests, connected, clearRequests }
}

```

8.2 Crea el hook de sesión `src/hooks/useSession.js`:

```

import { useState, useEffect } from 'react'

export function useSession() {
  const [token, setToken] = useState(null)

  useEffect(() => {
    let stored = localStorage.getItem('hooksuite_session')
    if (!stored) {
      stored = crypto.randomUUID()
      localStorage.setItem('hooksuite_session', stored)
    }
    setToken(stored)
  }, [])

  return token
}

```

Paso 9 — Crear el panel de peticiones interceptadas

9.1 Crea el componente `src/pages/proxy/RequestsTable.jsx`:

```

import { useState } from 'react'
import { Badge } from '@components/ui'

const METHOD_VARIANTS = {
  GET: 'get', POST: 'post', PUT: 'put', DELETE: 'delete',
}

const STATUS_COLOR = (status) => {

```

```

    if (status >= 200 && status < 300) return 'text-green-600'
    if (status >= 300 && status < 400) return 'text-blue-600'
    if (status >= 400 && status < 500) return 'text-yellow-600'
    return 'text-red-600'
  }

export function RequestsTable({ requests, onSelect, selectedId }) {
  const [filter, setFilter] = useState('')
  const [methodFilter, setMethodFilter] = useState('ALL')

  const filtered = requests.filter(r => {
    const matchesUrl = r.url.toLowerCase().includes(filter.toLowerCase())
    const matchesMethod = methodFilter === 'ALL' || r.method === methodFilter
    return matchesUrl && matchesMethod
  })

  return (
    <div className="flex flex-col h-full">
      <div className="flex gap-2 p-3 border-b border-gray-200 bg-gray-50">
        <input
          type="text"
          placeholder="Filtrar por URL..."
          value={filter}
          onChange={e => setFilter(e.target.value)}
          className="flex-1 px-3 py-1.5 text-sm border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-blue-500"
        />
        <select
          value={methodFilter}
          onChange={e => setMethodFilter(e.target.value)}
          className="px-3 py-1.5 text-sm border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-blue-500"
        >
          {[ 'ALL', 'GET', 'POST', 'PUT', 'DELETE', 'PATCH' ].map(m => (
            <option key={m} value={m}>{m}</option>
          ))}
        </select>
      </div>

      <div className="flex-1 overflow-auto">
        <table className="w-full text-sm">
          <thead className="bg-gray-50 border-b border-gray-200 sticky top-0">
            <tr>
              <th className="px-3 py-2 text-left text-xs font-medium text-gray-500
uppercase w-16">Método</th>
              <th className="px-3 py-2 text-left text-xs font-medium text-gray-500
uppercase">URL</th>
              <th className="px-3 py-2 text-left text-xs font-medium text-gray-500
uppercase w-16">Status</th>
              <th className="px-3 py-2 text-left text-xs font-medium text-gray-500
uppercase w-16">Tamaño</th>
              <th className="px-3 py-2 text-left text-xs font-medium text-gray-500
uppercase w-16">Tiempo</th>
            </tr>

```

```

</thead>
<tbody className="divide-y divide-gray-100">
  {filtered.map(req => (
    <tr
      key={req.id}
      onClick={() => onSelect(req)}
      className={`cursor-pointer transition-colors ${
        selectedId === req.id ? 'bg-blue-50' :
        req.vulnerable ? 'bg-red-50 hover:bg-red-100' :
        req.suspicious ? 'bg-yellow-50 hover:bg-yellow-100' :
        'hover:bg-gray-50'
      }`}
    >
      <td className="px-3 py-2">
        <Badge variant={METHOD_VARIANTS[req.method] || 'default'}>
{req.method}</Badge>
      </td>
      <td className="px-3 py-2 font-mono text-xs text-gray-700 max-w-xs
truncate">{req.url}</td>
      <td className={`px-3 py-2 font-medium
${STATUS_COLOR(req.status)}`}>{req.status}</td>
      <td className="px-3 py-2 text-gray-500">{(req.size /
1024).toFixed(1)}KB</td>
      <td className="px-3 py-2 text-gray-500">{req.time}ms</td>
    </tr>
  ))}
</tbody>
</table>
{filtered.length === 0 && (
  <div className="flex items-center justify-center h-32 text-gray-400
text-sm">
    No hay peticiones interceptadas
  </div>
)}
</div>
</div>
)
}

```

 **PROMPT DE IA** — Si necesitas ajustar la tabla o añadir columnas:

Tengo un componente React RequestsTable en HookSuite que muestra peticiones HTTP interceptadas en una tabla. Necesito añadir [describe qué necesitas: virtualización para muchas filas / ordenación por columna / exportar a CSV]. Dame el código para añadirlo al componente existente.

Paso 10 — Crear la vista detalle de petición

10.1 Crea el componente `src/pages/proxy/RequestDetail.jsx`:

```

import { useState } from 'react'
import { Button, Panel } from '@components/ui'

const TABS = ['Request', 'Response', 'Headers']

export function RequestDetail({ request, onSendToRepeater }) {
  const [activeTab, setActiveTab] = useState('Request')

  if (!request) {
    return (
      <div className="flex items-center justify-center h-full text-gray-400 text-sm">
        Selecciona una petición para ver el detalle
      </div>
    )
  }

  return (
    <div className="flex flex-col h-full">
      <div className="flex items-center justify-between px-4 py-3 border-b border-gray-200">
        <div className="flex gap-1">
          {TABS.map(tab => (
            <button
              key={tab}
              onClick={() => setActiveTab(tab)}
              className={`px-3 py-1.5 text-sm rounded-lg transition-colors ${
                activeTab === tab
                  ? 'bg-blue-600 text-white'
                  : 'text-gray-600 hover:bg-gray-100'
              }`}
            >
              {tab}
            </button>
          ))}
        </div>
        <Button size="sm" onClick={() => onSendToRepeater(request)}>
          Enviar al Repeater →
        </Button>
      </div>

      <div className="flex-1 overflow-auto p-4 font-mono text-xs">
        {activeTab === 'Request' && (
          <div>
            <div className="mb-4">
              <p className="text-blue-600 font-semibold mb-2">Petición</p>
              <p>{request.method} {request.url}</p>
            </div>
            {request.requestBody && (
              <div>
                <p className="text-blue-600 font-semibold mb-2">Body</p>
                <pre className="bg-gray-50 p-3 rounded-lg whitespace-pre-wrap

```

```

break-all">
        {request.requestBody}
      </pre>
    </div>
  )}
</div>
)}

{activeTab === 'Response' && (
  <div>
    <p className="text-green-600 font-semibold mb-2">Respuesta
{request.status}</p>
    <pre className="bg-gray-50 p-3 rounded-lg whitespace-pre-wrap break-
all text-gray-700">
      {request.responseBody}
    </pre>
  </div>
)}

{activeTab === 'Headers' && (
  <div className="space-y-4">
    <div>
      <p className="text-blue-600 font-semibold mb-2">Headers de
petición</p>
      {Object.entries(request.requestHeaders || {}).map(([k, v]) => (
        <div key={k} className="flex gap-2 py-1 border-b border-gray-100">
          <span className="text-gray-500 min-w-32">{k}:</span>
          <span className="text-gray-900 break-all">{v}</span>
        </div>
      ))}
    </div>
    <div>
      <p className="text-green-600 font-semibold mb-2">Headers de
respuesta</p>
      {Object.entries(request.responseHeaders || {}).map(([k, v]) => (
        <div key={k} className="flex gap-2 py-1 border-b border-gray-100">
          <span className="text-gray-500 min-w-32">{k}:</span>
          <span className="text-gray-900 break-all">{v}</span>
        </div>
      ))}
    </div>
  </div>
)}
</div>
</div>
)
}

```

Paso 11 — Crear el onboarding del PAC

11.1 Crea el componente `src/pages/proxy/PacOnboarding.jsx`:

```
import { useState, useEffect } from 'react'
import { Button } from '@components/ui'
import { config } from '@services/api'

const PAC_URL = `${config.API_BASE}/proxy.pac`

function detectBrowserOS() {
  const ua = navigator.userAgent
  const platform = navigator.platform
  if (ua.includes('Firefox')) return { browser: 'Firefox', os:
platform.includes('Win') ? 'Windows' : 'Mac' }
  if (ua.includes('Edg')) return { browser: 'Edge', os: 'Windows' }
  return { browser: 'Chrome', os: platform.includes('Win') ? 'Windows' : 'Mac' }
}

const INSTRUCTIONS = {
  'Chrome-Windows': [
    'Abre el menú de Chrome (tres puntos arriba a la derecha)',
    'Ve a Configuración → Sistema → Abrir configuración de proxy',
    'Activa "Usar script de configuración automática"',
    'Pega la URL del PAC en el campo de dirección',
    'Haz clic en Guardar',
  ],
  'Chrome-Mac': [
    'Abre Preferencias del Sistema → Red',
    'Selecciona tu conexión activa y haz clic en Avanzado',
    'Ve a la pestaña Proxies',
    'Activa "Configuración automática de proxy"',
    'Pega la URL del PAC y haz clic en Aceptar',
  ],
  'Firefox-Windows': [
    'Abre Ajustes → General → Configuración de red (al final)',
    'Selecciona "URL de configuración automática de proxy"',
    'Pega la URL del PAC',
    'Haz clic en Aceptar',
  ],
  'Firefox-Mac': [
    'Abre Preferencias → General → Configuración de red',
    'Selecciona "URL de configuración automática de proxy"',
    'Pega la URL del PAC y haz clic en Aceptar',
  ],
}

export function PacOnboarding({ onComplete }) {
  const [copied, setCopied] = useState(false)
  const [checking, setChecking] = useState(false)
  const [proxyActive, setProxyActive] = useState(false)
  const { browser, os } = detectBrowserOS()
  const instructions = INSTRUCTIONS[`${browser}-${os}`] || INSTRUCTIONS['Chrome-
Windows']

  const copyPacUrl = async () => {
```

```

    await navigator.clipboard.writeText(PAC_URL)
    setCopied(true)
    setTimeout(() => setCopied(false), 2000)
  }

  useEffect(() => {
    if (config.USE MOCKS) return
    setChecking(true)
    const interval = setInterval(async () => {
      try {
        const res = await fetch(`${config.API_BASE}/check/alive`)
        if (res.ok) {
          setProxyActive(true)
          clearInterval(interval)
          setTimeout(onComplete, 1500)
        }
      } catch {}
    }, 2000)
    return () => clearInterval(interval)
  }, [onComplete])

  return (
    <div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50">
      <div className="bg-white rounded-xl shadow-xl w-full max-w-lg p-6">
        <h2 className="text-lg font-semibold text-gray-900 mb-1">Configura el proxy</h2>
        <p className="text-sm text-gray-500 mb-4">
          Detectado: {browser} en {os}. Sigue estos pasos una sola vez.
        </p>

        <div className="bg-gray-50 rounded-lg p-3 mb-4">
          <p className="text-xs text-gray-500 mb-1">URL del archivo PAC</p>
          <div className="flex gap-2 items-center">
            <code className="flex-1 text-sm text-blue-600 break-all">{PAC_URL}</code>
            <Button size="sm" onClick={copyPacUrl} variant={copied ? 'secondary' : 'primary'}>
              {copied ? '✓ Copiado' : 'Copiar'}
            </Button>
          </div>
        </div>

        <ol className="space-y-2 mb-6">
          {instructions.map((step, i) => (
            <li key={i} className="flex gap-3 text-sm text-gray-700">
              <span className="flex-shrink-0 w-5 h-5 bg-blue-100 text-blue-700 rounded-full flex items-center justify-center text-xs font-medium">
                {i + 1}
              </span>
              {step}
            </li>
          ))}
        </ol>
      </div>
    </div>
  )

```

```

    {proxyActive ? (
      <div className="flex items-center gap-2 text-green-600 font-medium">
        <span>✓</span> Proxy activo. Cerrando...
      </div>
    ) : checking ? (
      <div className="text-sm text-gray-500">Verificando que el proxy está
activo...</div>
    ) : (
      <Button onClick={onComplete} variant="ghost" size="sm">
        Omitir por ahora
      </Button>
    )}
  </div>
</div>
)
}

```

Paso 12 — Ensamblar la página Proxy completa

12.1 Reemplaza el contenido de `src/pages/proxy/ProxyPage.jsx` con:

```

import { useState, useEffect } from 'react'
import { useSession } from '@/hooks/useSession'
import { useWebSocket } from '@/hooks/useWebSocket'
import { RequestsTable } from './RequestsTable'
import { RequestDetail } from './RequestDetail'
import { PacOnboarding } from './PacOnboarding'
import { ImportRequest } from './ImportRequest'
import { Button } from '@/components/ui'

export function ProxyPage() {
  const sessionToken = useSession()
  const { requests, connected, clearRequests } = useWebSocket(sessionToken)
  const [selectedRequest, setSelectedRequest] = useState(null)
  const [showOnboarding, setShowOnboarding] = useState(false)
  const [showImport, setShowImport] = useState(false)
  const [repeaterRequest, setRepeaterRequest] = useState(null)

  useEffect(() => {
    const seen = localStorage.getItem('hooksuite_pac_configured')
    if (!seen) setShowOnboarding(true)
  }, [])

  const handleOnboardingComplete = () => {
    localStorage.setItem('hooksuite_pac_configured', 'true')
    setShowOnboarding(false)
  }

  return (

```

```

<div className="flex flex-col h-full">
  <div className="flex items-center justify-between px-6 py-4 border-b border-
gray-200 bg-white">
    <div className="flex items-center gap-3">
      <h2 className="text-lg font-semibold text-gray-900">Proxy
Interceptor</h2>
      <span className={`inline-flex items-center gap-1.5 px-2.5 py-0.5
rounded-full text-xs font-medium ${
        connected ? 'bg-green-100 text-green-700' : 'bg-gray-100 text-gray-
500'
      }`}>
        <span className={`w-1.5 h-1.5 rounded-full ${connected ? 'bg-green-
500' : 'bg-gray-400'}`} />
        {connected ? 'Conectado' : 'Sin conexión'}
      </span>
    </div>
    <div className="flex gap-2">
      <Button size="sm" variant="ghost" onClick={() => setShowImport(true)}>
        Importar petición
      </Button>
      <Button size="sm" variant="ghost" onClick={clearRequests}>
        Limpiar
      </Button>
      <Button size="sm" variant="ghost" onClick={() =>
setShowOnboarding(true)}>
        Configurar proxy
      </Button>
    </div>
  </div>

  <div className="flex flex-1 overflow-hidden">
    <div className="w-1/2 border-r border-gray-200 overflow-hidden flex flex-
col">
      <RequestsTable
        requests={requests}
        onSelect={setSelectedRequest}
        selectedId={selectedRequest?.id}
      />
    </div>
    <div className="w-1/2 overflow-hidden flex flex-col">
      <RequestDetail
        request={selectedRequest}
        onSendToRepeater={(req) => {
          setRepeaterRequest(req)
          window.location.href = '/repeater'
        }}
      />
    </div>
  </div>

  {showOnboarding && <PacOnboarding onComplete={handleOnboardingComplete} />}
  {showImport && (
    <ImportRequest
      onClose={() => setShowImport(false)}

```

```

        onImport={({parsed}) => {
            setRepeaterRequest(parsed)
            setShowImport(false)
        }}
    />
    )}
</div>
)
}

```

12.2 Crea el componente `src/pages/proxy/ImportRequest.jsx`:

```

import { useState } from 'react'
import { Button } from '@components/ui'
import axios from 'axios'
import { config } from '@services/api'

export function ImportRequest({ onClose, onImport }) {
    const [text, setText] = useState('')
    const [loading, setLoading] = useState(false)
    const [error, setError] = useState(null)

    const handleParse = async () => {
        if (!text.trim()) return
        setLoading(true)
        setError(null)
        try {
            if (config.USE MOCKS) {
                onImport({ method: 'GET', url: 'http://example.com', headers: {}, body:
null })
                return
            }
            const res = await axios.post(`${config.API_BASE}/api/repeater/parse`, { text
})
            onImport(res.data)
        } catch (e) {
            setError('No se pudo parsear la petición. Verifica el formato.')
        } finally {
            setLoading(false)
        }
    }

    return (
        <div className="fixed inset-0 bg-black/50 flex items-center justify-center z-
50">
            <div className="bg-white rounded-xl shadow-xl w-full max-w-2xl p-6">
                <h3 className="text-lg font-semibold text-gray-900 mb-4">Importar
petición</h3>
                <p className="text-sm text-gray-500 mb-3">Pega una petición en formato raw
HTTP o cURL</p>
                <textarea
                    value={text}

```

```

      onChange={e => setText(e.target.value)}
      placeholder="GET /path HTTP/1.1&#10;Host: example.com&#10;Cookie:
session=abc&#10;&#10;o&#10;&#10;curl -X POST https://example.com -H 'Content-Type:
application/json' -d '{...}'"
      className="w-full h-48 px-3 py-2 text-sm font-mono border border-gray-
300 rounded-lg focus:outline-none focus:ring-2 focus:ring-blue-500 resize-none"
    />
    {error && <p className="text-sm text-red-600 mt-2">{error}</p>}
    <div className="flex gap-2 justify-end mt-4">
      <Button variant="ghost" onClick={onClose}>Cancelar</Button>
      <Button onClick={handleParse} disabled={loading}>
        {loading ? 'Parseando...' : 'Parsear y enviar al Repeater'}
      </Button>
    </div>
  </div>
</div>
)
}

```

12.3 Haz commit del bloque 02:

```

git add .
git commit -m "feat(frontend): módulo proxy interceptor completo con onboarding
PAC"
git push origin feature/frontend

```

BLOQUE 03 — Módulo Repeater

Paso 13 — Crear el editor de petición HTTP

🧡 **CONTACTO CON P2 Backend** — Para que el Repeater funcione con datos reales necesitas que P2 tenga listo el endpoint `POST /api/repeater/send`. Mientras tanto el componente usa mocks. Cuando P2 te confirme que está listo, cambia `USE MOCKS` a `false` en `api.js`.

13.1 Crea el componente `src/pages/repeater/RequestEditor.jsx`:

```

import { useState } from 'react'
import { Button } from '@components/ui'

export function RequestEditor({ initialRequest, onSend, loading }) {
  const [method, setMethod] = useState(initialRequest?.method || 'GET')
  const [url, setUrl] = useState(initialRequest?.url || '')
  const [headers, setHeaders] = useState(
    Object.entries(initialRequest?.requestHeaders || { 'Content-Type':
'application/json' })
      .map(([k, v]) => ({ key: k, value: v }))
  )

```

```

const [body, setBody] = useState(initialRequest?.requestBody || '')

const addHeader = () => setHeaders(prev => [...prev, { key: '', value: '' }])
const removeHeader = (i) => setHeaders(prev => prev.filter((_, idx) => idx !== i))
const updateHeader = (i, field, val) => setHeaders(prev =>
  prev.map((h, idx) => idx === i ? { ...h, [field]: val } : h)
)

const handleSend = () => {
  const headersObj = Object.fromEntries(headers.filter(h => h.key).map(h =>
    [h.key, h.value]))
  onSend({ method, url, headers: headersObj, body: body || null })
}

return (
  <div className="flex flex-col gap-4 p-4 h-full overflow-auto">
    <div className="flex gap-2">
      <select
        value={method}
        onChange={e => setMethod(e.target.value)}
        className="px-3 py-2 text-sm border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-blue-500"
      >
        {[ 'GET', 'POST', 'PUT', 'DELETE', 'PATCH', 'HEAD', 'OPTIONS' ].map(m => (
          <option key={m} value={m}>{m}</option>
        ))}
      </select>
      <input
        type="text"
        value={url}
        onChange={e => setUrl(e.target.value)}
        placeholder="https://objetivo.com/endpoint"
        className="flex-1 px-3 py-2 text-sm font-mono border border-gray-300
rounded-lg focus:outline-none focus:ring-2 focus:ring-blue-500"
      />
      <Button onClick={handleSend} disabled={loading || !url}>
        {loading ? 'Enviando...' : 'Enviar'}
      </Button>
    </div>

    <div>
      <div className="flex items-center justify-between mb-2">
        <p className="text-sm font-medium text-gray-700">Headers</p>
        <Button size="sm" variant="ghost" onClick={addHeader}>+ Añadir</Button>
      </div>
      <div className="space-y-1.5">
        {headers.map((h, i) => (
          <div key={i} className="flex gap-2">
            <input
              placeholder="Nombre"
              value={h.key}
              onChange={e => updateHeader(i, 'key', e.target.value)}
              className="w-1/3 px-2 py-1.5 text-xs font-mono border border-gray-

```

```

300 rounded focus:outline-none focus:ring-1 focus:ring-blue-500"
    />
    <input
      placeholder="Valor"
      value={h.value}
      onChange={e => updateHeader(i, 'value', e.target.value)}
      className="flex-1 px-2 py-1.5 text-xs font-mono border border-
gray-300 rounded focus:outline-none focus:ring-1 focus:ring-blue-500"
    />
    <button
      onClick={() => removeHeader(i)}
      className="text-gray-400 hover:text-red-500 px-1"
    >X</button>
  </div>
  )})
</div>
</div>

<div>
  <p className="text-sm font-medium text-gray-700 mb-2">Body</p>
  <textarea
    value={body}
    onChange={e => setBody(e.target.value)}
    placeholder="Body de la petición (JSON, form data, etc.)"
    className="w-full h-32 px-3 py-2 text-xs font-mono border border-gray-
300 rounded-lg focus:outline-none focus:ring-2 focus:ring-blue-500 resize-none"
  />
</div>
</div>
)
}

```

Paso 14 — Crear el panel de respuesta con diff

14.1 Crea el componente `src/pages/repeater/ResponsePanel.jsx`:

```

import { useState } from 'react'

const TABS = ['Raw', 'Pretty', 'Preview']

function StatusBadge({ status }) {
  const color = status >= 200 && status < 300 ? 'text-green-600 bg-green-50' :
    status >= 400 ? 'text-red-600 bg-red-50' : 'text-yellow-600 bg-
yellow-50'
  return <span className={`px-2 py-0.5 rounded text-sm font-medium ${color}`}>
{status}</span>
}

export function ResponsePanel({ response }) {
  const [tab, setTab] = useState('Pretty')

```

```

    if (!response) {
      return (
        <div className="flex items-center justify-center h-full text-gray-400 text-sm">
          Envía una petición para ver la respuesta aquí
        </div>
      )
    }

    return (
      <div className="flex flex-col h-full">
        <div className="flex items-center gap-4 px-4 py-2 border-b border-gray-200 bg-gray-50">
          <StatusBadge status={response.status} />
          <span className="text-xs text-gray-500">{response.time}ms</span>
          <span className="text-xs text-gray-500">{(response.size /
1024).toFixed(1)}KB</span>
          <div className="flex gap-1 ml-auto">
            {TABS.map(t => (
              <button
                key={t}
                onClick={() => setTab(t)}
                className={`px-3 py-1 text-xs rounded transition-colors ${
                  tab === t ? 'bg-blue-600 text-white' : 'text-gray-600 hover:bg-
gray-100'
                }`}
              >
                {t}
              </button>
            ))}
          </div>
        </div>
        <div className="flex-1 overflow-auto p-4">
          {tab === 'Raw' && (
            <pre className="text-xs font-mono text-gray-700 whitespace-pre-wrap break-all">
              {response.body}
            </pre>
          )}
          {tab === 'Pretty' && (
            <pre className="text-xs font-mono text-gray-700 whitespace-pre-wrap break-all bg-gray-50 p-3 rounded-lg">
              {(() => {
                try { return JSON.stringify(JSON.parse(response.body), null, 2) }
                catch { return response.body }
              })()}
            </pre>
          )}
          {tab === 'Preview' && (
            <iframe
              srcDoc={response.body}
              sandbox="allow-same-origin"
              className="w-full h-full border-0 rounded-lg"
            >

```

```

        title="Preview"
      />
    })
  </div>
</div>
)
}

```

14.2 Crea el comparador de respuestas `src/pages/repeater/DiffViewer.jsx`:

```

import { diffLines } from 'diff'

export function DiffViewer({ responseA, responseB }) {
  if (!responseA || !responseB) {
    return (
      <div className="flex items-center justify-center h-32 text-gray-400 text-sm">
        Necesitas al menos dos respuestas en el historial para comparar
      </div>
    )
  }

  const diffs = diffLines(responseA.body || '', responseB.body || '')

  return (
    <div className="flex gap-4 h-full overflow-auto p-4">
      <div className="flex-1">
        <p className="text-xs font-medium text-gray-500 mb-2">Respuesta A – {responseA.status}</p>
        <pre className="text-xs font-mono bg-gray-50 p-3 rounded-lg whitespace-pre-wrap break-all">
          {diffs.map((part, i) => (
            <span
              key={i}
              className={part.added ? 'bg-green-100 text-green-800' :
                part.removed ? 'bg-red-100 text-red-800' : 'text-gray-700'}
            >
              {part.value}
            </span>
          ))}
        </pre>
      </div>
    </div>
  )
}

```

15.1 Crea `src/pages/repeater/RepeaterPage.jsx`:

```
import { useState } from 'react'
import { RequestEditor } from '../RequestEditor'
import { ResponsePanel } from '../ResponsePanel'
import { Button } from '@components/ui'
import axios from 'axios'
import { config } from '@services/api'

const MOCK_RESPONSE = {
  status: 200,
  time: 134,
  size: 2048,
  body: JSON.stringify({ message: 'OK', data: { id: 1, name: 'admin' } }, null,
2),
  headers: { 'Content-Type': 'application/json' },
}

export function RepeaterPage() {
  const [response, setResponse] = useState(null)
  const [loading, setLoading] = useState(false)
  const [history, setHistory] = useState([])
  const [compareMode, setCompareMode] = useState(false)

  const handleSend = async (request) => {
    setLoading(true)
    try {
      if (config.USE MOCKS) {
        await new Promise(r => setTimeout(r, 500))
        setResponse(MOCK_RESPONSE)
        setHistory(prev => [{ request, response: MOCK_RESPONSE, timestamp: new
Date() }, ...prev.slice(0, 99)])
        return
      }
      const res = await axios.post(`${config.API_BASE}/api/repeater/send`,
request)
      setResponse(res.data)
      setHistory(prev => [{ request, response: res.data, timestamp: new Date() },
...prev.slice(0, 99)])
    } catch (e) {
      setResponse({ status: 0, time: 0, size: 0, body: 'Error: ' + e.message })
    } finally {
      setLoading(false)
    }
  }

  return (
    <div className="flex flex-col h-full">
      <div className="flex items-center justify-between px-6 py-4 border-b border-
gray-200 bg-white">
        <h2 className="text-lg font-semibold text-gray-900">Repeater</h2>
        <Button size="sm" variant="ghost" onClick={() =>
setCompareMode(!compareMode)}>

```

```

        {compareMode ? 'Vista normal' : 'Comparar respuestas'}
      </Button>
    </div>
    <div className="flex flex-1 overflow-hidden">
      <div className="w-8 bg-gray-50 border-r border-gray-200 flex flex-col py-2
overflow-auto">
        {history.map((item, i) => (
          <button
            key={i}
            title={`${item.request.method} ${item.request.url}`}
            className="w-full px-2 py-1.5 text-xs text-gray-500 hover:bg-gray-
100 text-left truncate"
          >
            {i + 1}
          </button>
        ))}
      </div>
      <div className="flex-1 flex overflow-hidden">
        <div className="w-1/2 border-r border-gray-200 overflow-hidden flex
flex-col">
          <RequestEditor onSend={handleSend} loading={loading} />
        </div>
        <div className="w-1/2 overflow-hidden flex flex-col">
          <ResponsePanel response={response} />
        </div>
      </div>
    </div>
  </div>
)
}

```

15.2 Commit del bloque 03:

```

git add .
git commit -m "feat(frontend): módulo repeater con editor, panel de respuesta y
diff"
git push origin feature/frontend

```

BLOQUE 04 — Módulo Intruder

Paso 16 — Crear la configuración de ataque

📌 **CONTACTO CON P2 Backend** — Para este módulo necesitas que P2 tenga listos tres endpoints:

- `POST /api/intruder/start` — Inicia el ataque
- `POST /api/intruder/pause/{token}` — Pausa el ataque
- `POST /api/intruder/cancel/{token}` — Cancela el ataque
- `GET /api/payloads/{tipo}` — Devuelve la lista de payloads

El progreso del ataque llega por WebSocket. Asegúrate de que P2 emite eventos con el formato: { type: 'intruder_result', payload: { payload, status, size, time, vulnerable } }

16.1 Crea src/pages/intruder/IntruderPage.jsx:

```
import { useState } from 'react'
import { Button, Badge } from '@components/ui'
import axios from 'axios'
import { config } from '@services/api'
import { useSession } from '@hooks/useSession'

const ATTACK_TYPES = [
  { id: 'sqli', label: 'SQL Injection' },
  { id: 'blind_sqli', label: 'Blind SQLi' },
  { id: 'xss', label: 'XSS' },
  { id: 'fuzzing', label: 'Fuzzing genérico' },
]

const MOCK_RESULTS = [
  { id: 1, payload: "' OR '1'='1", status: 200, size: 8934, time: 312, vulnerable: true },
  { id: 2, payload: "' OR '1'='2", status: 200, size: 4823, time: 89, vulnerable: false },
  { id: 3, payload: "1 UNION SELECT null--", status: 500, size: 1024, time: 156, vulnerable: true },
  { id: 4, payload: "1; DROP TABLE users--", status: 200, size: 4823, time: 91, vulnerable: false },
]

export function IntruderPage() {
  const sessionToken = useSession()
  const [url, setUrl] = useState('')
  const [injectionPoint, setInjectionPoint] = useState('')
  const [attackType, setAttackType] = useState('sqli')
  const [results, setResults] = useState([])
  const [running, setRunning] = useState(false)
  const [progress, setProgress] = useState(0)
  const [total, setTotal] = useState(0)

  const handleStart = async () => {
    setResults([])
    setRunning(true)
    setProgress(0)

    if (config.USE MOCKS) {
      setTotal(MOCK_RESULTS.length)
      for (let i = 0; i < MOCK_RESULTS.length; i++) {
        await new Promise(r => setTimeout(r, 400))
        setResults(prev => [...prev, MOCK_RESULTS[i]])
        setProgress(i + 1)
      }
      setRunning(false)
      return
    }
  }
}
```

```

    }

    try {
      await axios.post(`${config.API_BASE}/api/intruder/start`, {
        url, injectionPoint, attackType, sessionToken
      })
    } catch (e) {
      setRunning(false)
    }
  }
}

const handleCancel = async () => {
  if (!config.USE MOCKS) {
    await axios.post(`${config.API_BASE}/api/intruder/cancel/${sessionToken}`)
  }
  setRunning(false)
}

return (
  <div className="flex flex-col h-full">
    <div className="px-6 py-4 border-b border-gray-200 bg-white">
      <h2 className="text-lg font-semibold text-gray-900">Intruder</h2>
    </div>

    <div className="flex flex-1 overflow-hidden">
      <div className="w-80 border-r border-gray-200 p-4 overflow-auto bg-gray-50">
        <div className="space-y-4">
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">URL
objetivo</label>
            <input
              value={url}
              onChange={e => setUrl(e.target.value)}
              placeholder="http://dvwa.local/sqli/?id=1"
              className="w-full px-3 py-2 text-sm font-mono border border-gray-300 rounded-lg focus:outline-none focus:ring-2 focus:ring-blue-500"
            />
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">Punto de inyección</label>
            <input
              value={injectionPoint}
              onChange={e => setInjectionPoint(e.target.value)}
              placeholder="id (el parámetro a inyectar)"
              className="w-full px-3 py-2 text-sm border border-gray-300 rounded-lg focus:outline-none focus:ring-2 focus:ring-blue-500"
            />
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">Tipo

```

```

de ataque</label>
    <select
      value={attackType}
      onChange={e => setAttackType(e.target.value)}
      className="w-full px-3 py-2 text-sm border border-gray-300
rounded-lg focus:outline-none focus:ring-2 focus:ring-blue-500"
    >
      {ATTACK_TYPES.map(t => <option key={t.id} value={t.id}>{t.label}
</option>)}
    </select>
  </div>

  {running ? (
    <div className="space-y-2">
      <div className="w-full bg-gray-200 rounded-full h-2">
        <div
          className="bg-blue-600 h-2 rounded-full transition-all"
          style={{ width: total ? `${(progress / total) * 100}%` : '0%'
}}
        />
      </div>
      <p className="text-xs text-gray-500">{progress} / {total || '?'}</p>
      <Button variant="danger" onClick={handleCancel} className="w-
full">
        Cancelar ataque
      </Button>
    </div>
  ) : (
    <Button onClick={handleStart} className="w-full" disabled={!url}>
      Iniciar ataque
    </Button>
  )}
</div>
</div>

<div className="flex-1 overflow-auto">
  <table className="w-full text-sm">
    <thead className="bg-gray-50 border-b border-gray-200 sticky top-0">
      <tr>
        <th className="px-4 py-2 text-left text-xs font-medium text-gray-
500 uppercase">#</th>
        <th className="px-4 py-2 text-left text-xs font-medium text-gray-
500 uppercase">Payload</th>
        <th className="px-4 py-2 text-left text-xs font-medium text-gray-
500 uppercase">Status</th>
        <th className="px-4 py-2 text-left text-xs font-medium text-gray-
500 uppercase">Tamaño</th>
        <th className="px-4 py-2 text-left text-xs font-medium text-gray-
500 uppercase">Tiempo</th>
        <th className="px-4 py-2 text-left text-xs font-medium text-gray-
500 uppercase">IA</th>
      </tr>
    </thead>

```

```

<tbody className="divide-y divide-gray-100">
  {results.map((r) => (
    <tr key={r.id} className={r.vulnerable ? 'bg-red-50' : 'hover:bg-
gray-50'}>
      <td className="px-4 py-2 text-gray-500">{r.id}</td>
      <td className="px-4 py-2 font-mono text-xs">{r.payload}</td>
      <td className="px-4 py-2">{r.status}</td>
      <td className="px-4 py-2 text-gray-500">{r.size}B</td>
      <td className="px-4 py-2 text-gray-500">{r.time}ms</td>
      <td className="px-4 py-2">
        {r.vulnerable
          ? <Badge variant="critical">Vulnerable</Badge>
          : <Badge variant="default">Limpio</Badge>
        }
      </td>
    </tr>
  ))}
</tbody>
</table>
{results.length === 0 && !running && (
  <div className="flex items-center justify-center h-32 text-gray-400
text-sm">
    Configura un ataque y haz clic en Iniciar
  </div>
)}
</div>
</div>
</div>
)
}

```

16.2 Commit del bloque 04:

```

git add .
git commit -m "feat(frontend): módulo intruder con configuración y tabla de
resultados"
git push origin feature/frontend

```

 **PROMPT DE IA** — Si necesitas añadir la selección visual del punto de inyección en un editor de texto:

Estoy construyendo el módulo Intruder de HookSuite. Necesito un componente React donde el usuario pegue una petición HTTP y pueda seleccionar con el ratón el fragmento que quiere usar como punto de inyección. El fragmento seleccionado se marca visualmente y se guarda en el estado. Dame el componente completo con Tailwind CSS.

BLOQUE 05 — Panel de vulnerabilidades y red en tiempo real

Paso 17 — Crear el dashboard de vulnerabilidades

📖 **CONTACTO CON P5 IA** — Las vulnerabilidades llegan desde el módulo de IA a través de P2 por WebSocket. El evento tiene el formato: `{ type: 'vulnerability_detected', payload: { id, tipo, severidad, titulo, descripcion, url, payload, recomendacion, timestamp } }`. Actualiza el hook `useWebSocket` para escuchar también este tipo de evento cuando P5 te confirme que está emitiendo.

17.1 Crea `src/pages/vulnerabilities/VulnerabilitiesPage.jsx`:

```
import { useState } from 'react'
import { Badge, Panel } from '@components/ui'
import { mockVulnerabilities } from '@services/mockData'
import { config } from '@services/api'

const SEVERITY_ORDER = { critical: 0, high: 1, medium: 2, low: 3 }

export function VulnerabilitiesPage() {
  const [vulnerabilities] = useState(config.USE MOCKS ? mockVulnerabilities : [])
  const [selected, setSelected] = useState(null)
  const [filter, setFilter] = useState('all')

  const filtered = vulnerabilities
    .filter(v => filter === 'all' || v.severidad === filter)
    .sort((a, b) => (SEVERITY_ORDER[a.severidad] || 99) -
      (SEVERITY_ORDER[b.severidad] || 99))

  return (
    <div className="flex flex-col h-full">
      <div className="flex items-center justify-between px-6 py-4 border-b border-gray-200 bg-white">
        <div className="flex items-center gap-3">
          <h2 className="text-lg font-semibold text-gray-900">Vulnerabilidades</h2>
          <span className="text-sm text-gray-500">{vulnerabilities.length}</span>
        </div>
        <select
          value={filter}
          onChange={e => setFilter(e.target.value)}
          className="px-3 py-1.5 text-sm border border-gray-300 rounded-lg focus:outline-none"
        >
          <option value="all">Todas</option>
          <option value="critical">Crítica</option>
          <option value="high">Alta</option>
          <option value="medium">Media</option>
          <option value="low">Baja</option>
        </select>
      </div>
    </div>
  )
}
```

```

</div>

<div className="flex flex-1 overflow-hidden">
  <div className="w-1/2 border-r border-gray-200 overflow-auto">
    {filtered.map(v => (
      <div
        key={v.id}
        onClick={() => setSelected(v)}
        className={`p-4 border-b border-gray-100 cursor-pointer transition-
colors ${
          selected?.id === v.id ? 'bg-blue-50' : 'hover:bg-gray-50'
        }}`}
      >
        <div className="flex items-start justify-between gap-2">
          <div className="flex-1 min-w-0">
            <p className="text-sm font-medium text-gray-900 truncate">
{v.titulo}</p>
            <p className="text-xs text-gray-500 mt-0.5 truncate font-mono">
{v.url}</p>
          </div>
          <Badge variant={v.severidad}>
            {v.severidad === 'critical' ? 'Crítica' :
              v.severidad === 'high' ? 'Alta' :
              v.severidad === 'medium' ? 'Media' : 'Baja'}
          </Badge>
        </div>
        <p className="text-xs text-gray-500 mt-1">{v.tipo}</p>
      </div>
    ))}
    {filtered.length === 0 && (
      <div className="flex items-center justify-center h-32 text-gray-400
text-sm">
        No se han detectado vulnerabilidades todavía
      </div>
    )}
  </div>

  <div className="w-1/2 overflow-auto p-4">
    {selected ? (
      <div className="space-y-4">
        <div>
          <div className="flex items-center gap-2 mb-1">
            <Badge variant={selected.severidad}>{selected.severidad}</Badge>
            <span className="text-xs text-gray-500">{selected.tipo}</span>
          </div>
          <h3 className="text-base font-semibold text-gray-900">
{selected.titulo}</h3>
          <p className="text-xs font-mono text-gray-500 mt-0.5">
{selected.url}</p>
        </div>
        <div>
          <p className="text-xs font-medium text-gray-500 uppercase mb-
1">Descripción</p>
          <p className="text-sm text-gray-700">{selected.descripcion}</p>

```

```

        </div>
        <div>
          <p className="text-xs font-medium text-gray-500 uppercase mb-1">Payload usado</p>
          <code className="block text-xs bg-gray-50 p-2 rounded font-mono text-red-700">
            {selected.payload}
          </code>
        </div>
        <div>
          <p className="text-xs font-medium text-gray-500 uppercase mb-1">Recomendación</p>
          <p className="text-sm text-gray-700">{selected.recomendacion}</p>
        </div>
      </div>
    ) : (
      <div className="flex items-center justify-center h-full text-gray-400 text-sm">
        Selecciona una vulnerabilidad para ver el detalle
      </div>
    )}
  </div>
</div>
</div>
)
}

```

Paso 18 — Crear el panel de red en tiempo real

🔥 **CONTACTO CON P4 DevTools** — Los paquetes de red llegan de P4 a través de P2 por WebSocket. El evento tiene el formato acordado el día 1. Actualiza el hook `useWebSocket` para escuchar el tipo de evento `network_packet` cuando P4 te confirme que está emitiendo.

18.1 Crea `src/pages/network/NetworkPage.jsx`:

```

import { useState } from 'react'
import { Badge } from '@components/ui'
import { mockRequests } from '@services/mockData'
import { config } from '@services/api'

export function NetworkPage() {
  const [packets] = useState(config.USE MOCKS ? mockRequests : [])

  return (
    <div className="flex flex-col h-full">
      <div className="px-6 py-4 border-b border-gray-200 bg-white">
        <h2 className="text-lg font-semibold text-gray-900">Red en tiempo
real</h2>
        <p className="text-sm text-gray-500 mt-0.5">Tráfico capturado por Chrome
DevTools</p>

```

```

</div>

<div className="flex-1 overflow-auto">
  <table className="w-full text-sm">
    <thead className="bg-gray-50 border-b border-gray-200 sticky top-0">
      <tr>
        <th className="px-4 py-2 text-left text-xs font-medium text-gray-500 uppercase">Estado</th>
        <th className="px-4 py-2 text-left text-xs font-medium text-gray-500 uppercase">Método</th>
        <th className="px-4 py-2 text-left text-xs font-medium text-gray-500 uppercase">URL</th>
        <th className="px-4 py-2 text-left text-xs font-medium text-gray-500 uppercase">Status</th>
        <th className="px-4 py-2 text-left text-xs font-medium text-gray-500 uppercase">Tiempo</th>
      </tr>
    </thead>
    <tbody className="divide-y divide-gray-100">
      {packets.map(p => (
        <tr
          key={p.id}
          className={
            p.vulnerable ? 'bg-red-50' :
            p.suspicious ? 'bg-yellow-50' :
            'hover:bg-gray-50'
          }
        >
          <td className="px-4 py-2">
            {p.vulnerable
              ? <Badge variant="critical">Vulnerable</Badge>
              : p.suspicious
              ? <Badge variant="warning">Sospechoso</Badge>
              : <Badge variant="default">Limpio</Badge>
            }
          </td>
          <td className="px-4 py-2">
            <Badge variant={p.method?.toLowerCase() || 'default'}>{p.method}</Badge>
          </td>
          <td className="px-4 py-2 font-mono text-xs text-gray-700 max-w-xs truncate">{p.url}</td>
          <td className="px-4 py-2">{p.status}</td>
          <td className="px-4 py-2 text-gray-500">{p.time}ms</td>
        </tr>
      )))
    </tbody>
  </table>
  {packets.length === 0 && (
    <div className="flex items-center justify-center h-32 text-gray-400 text-sm">
      Esperando tráfico de red...
    </div>
  )}

```

```

        </div>
      </div>
    )
  }

```

18.2 Commit del bloque 05:

```

git add .
git commit -m "feat(frontend): panel de vulnerabilidades y panel de red en tiempo real"
git push origin feature/frontend

```

BLOQUE 06 — Módulo de Utilidades

Paso 19 — Crear el módulo de utilidades completo

🟡 **CONTACTO CON P2 Backend** — El Hash Generator necesita el endpoint `POST /api/utils/hash`. El Regex Tester puede funcionar completamente en el frontend con JavaScript nativo. El Encoder/Decoder también funciona en el frontend sin llamar al backend.

19.1 Crea `src/pages/utilities/UtilitiesPage.jsx`:

```

import { useState } from 'react'
import { EncoderDecoder } from './EncoderDecoder'
import { HashGenerator } from './HashGenerator'
import { RegexTester } from './RegexTester'
import { PayloadGenerator } from './PayloadGenerator'

const TOOLS = [
  { id: 'encoder', label: 'Encoder / Decoder' },
  { id: 'hash', label: 'Hash Generator' },
  { id: 'regex', label: 'Regex Tester' },
  { id: 'payloads', label: 'Payload Generator' },
]

export function UtilitiesPage() {
  const [activeTool, setActiveTool] = useState('encoder')

  return (
    <div className="flex flex-col h-full">
      <div className="px-6 py-4 border-b border-gray-200 bg-white">
        <h2 className="text-lg font-semibold text-gray-900">Utilidades</h2>
      </div>
      <div className="flex flex-1 overflow-hidden">
        <div className="w-48 border-r border-gray-200 bg-gray-50 p-3">
          <nav className="space-y-1">
            {TOOLS.map(t => (

```

```

        <button
          key={t.id}
          onClick={() => setActiveTool(t.id)}
          className={`w-full text-left px-3 py-2 text-sm rounded-lg
transition-colors ${
            activeTool === t.id
              ? 'bg-blue-600 text-white'
              : 'text-gray-700 hover:bg-gray-100'
            }`}
        >
          {t.label}
        </button>
      )}
    </nav>
  </div>
  <div className="flex-1 overflow-auto p-6">
    {activeTool === 'encoder' && <EncoderDecoder />}
    {activeTool === 'hash' && <HashGenerator />}
    {activeTool === 'regex' && <RegexTester />}
    {activeTool === 'payloads' && <PayloadGenerator />}
  </div>
</div>
</div>
)
}

```

19.2 Crea `src/pages/utilities/EncoderDecoder.jsx`:

```

import { useState } from 'react'

function encodeHTML(str) {
  return
  str.replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;').replace(/"/g, '&quot;')
}

function decodeJWT(token) {
  try {
    const parts = token.split('.')
    if (parts.length !== 3) return 'JWT inválido'
    const header = JSON.parse(atob(parts[0]))
    const payload = JSON.parse(atob(parts[1].replace(/-/g, '+').replace(/_/g, '/')))
    return JSON.stringify({ header, payload }, null, 2)
  } catch { return 'No se pudo decodificar el JWT' }
}

export function EncoderDecoder() {
  const [input, setInput] = useState('')

  const outputs = [
    { label: 'Base64 encode', value: (() => { try { return btoa(input) } catch {

```

```

return 'Error' } } ) },
  { label: 'Base64 decode', value: (() => { try { return atob(input) } catch {
return 'No es Base64 válido' } } ) },
  { label: 'URL encode', value: encodeURIComponent(input) },
  { label: 'URL decode', value: (() => { try { return decodeURIComponent(input)
} catch { return 'Error' } } ) },
  { label: 'HTML encode', value: encodeHTML(input) },
  { label: 'JWT decode', value: input.includes('.') ? decodeJWT(input) :
'Introduce un JWT' },
]

return (
  <div className="space-y-4">
    <div>
      <label className="block text-sm font-medium text-gray-700 mb-1">Texto de
entrada</label>
      <textarea
        value={input}
        onChange={e => setInput(e.target.value)}
        placeholder="Introduce el texto a transformar..."
        className="w-full h-24 px-3 py-2 text-sm font-mono border border-gray-
300 rounded-lg focus:outline-none focus:ring-2 focus:ring-blue-500 resize-none"
      />
    </div>
    <div className="grid grid-cols-2 gap-3">
      {outputs.map(o => (
        <div key={o.label} className="border border-gray-200 rounded-lg p-3">
          <div className="flex items-center justify-between mb-1">
            <p className="text-xs font-medium text-gray-500">{o.label}</p>
            <button
              onClick={() => navigator.clipboard.writeText(o.value)}
              className="text-xs text-blue-600 hover:text-blue-800"
            >
              Copiar
            </button>
          </div>
          <pre className="text-xs font-mono text-gray-700 whitespace-pre-wrap
break-all max-h-20 overflow-auto">
            {o.value || '-'}
          </pre>
        </div>
      ))}
    </div>
  </div>
)
}

```

19.3 Crea `src/pages/utilities/HashGenerator.jsx`:

```

import { useState } from 'react'
import { Button } from '@components/ui'
import axios from 'axios'

```

```

import { config } from '@services/api'

const MOCK_HASHES = {
  md5: '5f4dcc3b5aa765d61d8327deb882cf99',
  sha1: '5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8',
  sha256: '5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8',
  sha512:
    'b109f3bbbc244eb82441917ed06d618b9008dd09b3befd1b5e07394c706a8bb980b1d7785e5976ec0
    49b46df5f1326af5a2ea6d103fd07c95385ffab0cacbc86',
}

export function HashGenerator() {
  const [input, setInput] = useState('')
  const [hashes, setHashes] = useState(null)
  const [loading, setLoading] = useState(false)

  const generate = async () => {
    if (!input) return
    setLoading(true)
    try {
      if (config.USE MOCKS) {
        await new Promise(r => setTimeout(r, 300))
        setHashes(MOCK_HASHES)
        return
      }
      const res = await axios.post(`${config.API_BASE}/api/utils/hash`, { text:
input })
      setHashes(res.data)
    } finally {
      setLoading(false)
    }
  }

  return (
    <div className="space-y-4">
      <div className="flex gap-2">
        <input
          value={input}
          onChange={e => setInput(e.target.value)}
          placeholder="Texto a hashear..."
          className="flex-1 px-3 py-2 text-sm font-mono border border-gray-300
rounded-lg focus:outline-none focus:ring-2 focus:ring-blue-500"
          onKeyDown={e => e.key === 'Enter' && generate()}
        />
        <Button onClick={generate} disabled={loading || !input}>
          {loading ? 'Generando...' : 'Generar'}
        </Button>
      </div>
      {hashes && (
        <div className="space-y-2">
          {Object.entries(hashes).map(([algo, hash]) => (
            <div key={algo} className="border border-gray-200 rounded-lg p-3">
              <div className="flex items-center justify-between mb-1">
                <p className="text-xs font-medium text-gray-500 uppercase">{algo}

```

```

    </p>
    <button
      onClick={() => navigator.clipboard.writeText(hash)}
      className="text-xs text-blue-600 hover:text-blue-800"
    >
      Copiar
    </button>
  </div>
  <p className="text-xs font-mono text-gray-700 break-all">{hash}</p>
</div>
  )})
</div>
  )
}

```

19.4 Crea `src/pages/utilities/RegexTester.jsx`:

```

import { useState, useMemo } from 'react'

export function RegexTester() {
  const [pattern, setPattern] = useState('')
  const [flags, setFlags] = useState('g')
  const [text, setText] = useState('')

  const { matches, error, highlighted } = useMemo(() => {
    if (!pattern || !text) return { matches: [], error: null, highlighted: text }
    try {
      const regex = new RegExp(pattern, flags)
      const found = [...text.matchAll(new RegExp(pattern, flags.includes('g') ?
flags : flags + 'g'))]
      const parts = []
      let last = 0
      found.forEach(m => {
        if (m.index > last) parts.push({ text: text.slice(last, m.index), match:
false })
        parts.push({ text: m[0], match: true })
        last = m.index + m[0].length
      })
      if (last < text.length) parts.push({ text: text.slice(last), match: false })
      return { matches: found, error: null, highlighted: parts }
    } catch (e) {
      return { matches: [], error: e.message, highlighted: [] }
    }
  }, [pattern, flags, text])

  return (
    <div className="space-y-4">
      <div className="flex gap-2">
        <div className="flex-1">
          <label className="block text-sm font-medium text-gray-700 mb-1">Patrón

```

```

regex</label>
  <input
    value={pattern}
    onChange={e => setPattern(e.target.value)}
    placeholder="([a-z+)\d+"
    className={`w-full px-3 py-2 text-sm font-mono border rounded-lg
focus:outline-none focus:ring-2 ${
      error ? 'border-red-300 focus:ring-red-500' : 'border-gray-300
focus:ring-blue-500'
    }`}
  />
  {error && <p className="text-xs text-red-600 mt-1">{error}</p>}
</div>
<div>
  <label className="block text-sm font-medium text-gray-700 mb-
1">Flags</label>
  <input
    value={flags}
    onChange={e => setFlags(e.target.value)}
    className="w-20 px-3 py-2 text-sm font-mono border border-gray-300
rounded-lg focus:outline-none focus:ring-2 focus:ring-blue-500"
  />
</div>
</div>

<div>
  <label className="block text-sm font-medium text-gray-700 mb-1">Texto de
prueba</label>
  <textarea
    value={text}
    onChange={e => setText(e.target.value)}
    placeholder="Texto donde buscar el patrón..."
    className="w-full h-32 px-3 py-2 text-sm font-mono border border-gray-
300 rounded-lg focus:outline-none focus:ring-2 focus:ring-blue-500 resize-none"
  />
</div>

{Array.isArray(highlighted) && highlighted.length > 0 && (
  <div>
    <p className="text-sm font-medium text-gray-700 mb-1">
      Resultado – {matches.length} match{matches.length !== 1 ? 'es' : ''}
    </p>
    <div className="bg-gray-50 p-3 rounded-lg text-sm font-mono whitespace-
pre-wrap break-all">
      {highlighted.map((part, i) => (
        <span key={i} className={part.match ? 'bg-yellow-200 text-yellow-900
rounded' : ''}>
          {part.text}
        </span>
      ))}
    </div>
  </div>
)}
</div>

```

```
)
}
```

19.5 Crea `src/pages/utilities/PayloadGenerator.jsx`:

```
import { useState } from 'react'
import { Button } from '@components/ui'

const PAYLOADS = {
  sqli: ["' OR '1'='1", "' OR '1'='1'--", "1 UNION SELECT null--", "' AND 1=0 UNION SELECT null,null--", "admin'--", "1; DROP TABLE users--"],
  blind_sqli: ["1' AND SUBSTRING((SELECT database()),1,1)='a'--", "1' AND 1=1--", "1' AND 1=2--", "1' AND LENGTH(database())>1--"],
  xss: ['<script>alert(1)</script>', '<img src=x onerror=alert(1)>', '<script>alert(1)</script>', "javascript:alert(1)", '<svg onload=alert(1)>'],
  fuzzing: ['"', "'", '<', '>', '&', ';', '|', '`', ' ../', ' ../ ../etc/passwd', '%00', 'null', 'undefined', '${7*7}'],
}

const LABELS = { sqli: 'SQL Injection', blind_sqli: 'Blind SQLi', xss: 'XSS', fuzzing: 'Fuzzing genérico' }

export function PayloadGenerator() {
  const [type, setType] = useState('sqli')
  const [copied, setCopied] = useState(null)

  const copyAll = () => {
    navigator.clipboard.writeText(PAYLOADS[type].join('\n'))
    setCopied('all')
    setTimeout(() => setCopied(null), 2000)
  }

  return (
    <div className="space-y-4">
      <div className="flex items-center gap-3">
        <select
          value={type}
          onChange={e => setType(e.target.value)}
          className="px-3 py-2 text-sm border border-gray-300 rounded-lg focus:outline-none focus:ring-2 focus:ring-blue-500"
        >
          {Object.entries(LABELS).map(([k, v]) => <option key={k} value={k}>{v}</option>)}
        </select>
        <Button size="sm" variant="secondary" onClick={copyAll}>
          {copied === 'all' ? '✓ Copiado todo' : 'Copiar todos'}
        </Button>
      </div>

      <div className="space-y-1.5">
        {PAYLOADS[type].map((payload, i) => (
          <div key={i} className="flex items-center gap-2 bg-gray-50 border
```

```
border-gray-200 rounded-lg px-3 py-2">
  <code className="flex-1 text-xs font-mono text-gray-700">{payload}
</code>
  <button
    onClick={() => { navigator.clipboard.writeText(payload);
setCopied(i); setTimeout(() => setCopied(null), 1500) }}
    className="text-xs text-blue-600 hover:text-blue-800 whitespace-
nowrap"
  >
    {copied === i ? '✓' : 'Copiar'}
  </button>
</div>
))}
</div>
</div>
)
}
```

19.6 Commit del bloque 06:

```
git add .
git commit -m "feat(frontend): módulo de utilidades completo (encoder, hash,
regex, payloads)"
git push origin feature/frontend
```

BLOQUE 07 — Integración con datos reales y UX final

Paso 20 — Conectar con el backend de P2

Cuando P2 te confirme que el backend está listo en local, sigue estos pasos:

📌 CONTACTO CON P2 Backend — Necesitas que P2 te confirme:

1. La URL del servidor en local (por ejemplo `http://localhost:8000`)
2. Que el endpoint `/health` responde correctamente
3. Que el WebSocket en `/ws/{token}` acepta conexiones

Una vez que lo confirmes, haz los cambios de los pasos siguientes.

20.1 Abre `src/services/api.js` y cambia `USE MOCKS` a `false`:

```
const USE MOCKS = false
```

20.2 Crea un archivo `.env.local` en la raíz del proyecto frontend (este archivo no se sube a GitHub):

```
VITE_API_URL=http://localhost:8000
```

20.3 Arranca el frontend y verifica que la conexión WebSocket se establece correctamente:

```
npm run dev
```

Abre la consola del navegador (F12) y busca errores de conexión WebSocket. Si hay errores, compártelos con P2 para que los resuelva.

 **PROMPT DE IA** — Si tienes errores de conexión WebSocket o CORS:

Estoy conectando el frontend React de HookSuite con el backend FastAPI. El frontend usa WebSocket en `ws://localhost:8000/ws/{token}`. Tengo este error en la consola: [pega el error exacto]. ¿Qué puede estar fallando y cómo lo soluciono desde el frontend?

Paso 21 — Pulir la UX: estados de carga y errores

21.1 Crea un componente de estado vacío reutilizable `src/components/ui/EmptyState.jsx`:

```
export function EmptyState({ title, description }) {
  return (
    <div className="flex flex-col items-center justify-center h-full py-12 text-center">
      <div className="w-12 h-12 bg-gray-100 rounded-full flex items-center justify-center mb-3">
        <span className="text-gray-400 text-xl">○</span>
      </div>
      <p className="text-sm font-medium text-gray-900">{title}</p>
      {description} && <p className="text-xs text-gray-500 mt-1 max-w-xs">
        {description}</p>
      </div>
    )
  }
}
```

21.2 Crea un componente de error `src/components/ui/ErrorMessage.jsx`:

```
export function ErrorMessage({ message, onRetry }) {
  return (
    <div className="flex items-center gap-3 p-3 bg-red-50 border border-red-200 rounded-lg text-sm">
      <span className="text-red-500">⚠️</span>
      <p className="flex-1 text-red-700">{message}</p>
    </div>
  )
}
```

```
    {onRetry && (
      <button onClick={onRetry} className="text-red-600 hover:text-red-800
underline text-xs">
        Reintentar
      </button>
    )}
  </div>
)
```

21.3 Añade los nuevos componentes al índice `src/components/ui/index.js`:

```
export { Button } from './Button'
export { Badge } from './Badge'
export { Spinner } from './Spinner'
export { Panel } from './Panel'
export { EmptyState } from './EmptyState'
export { ErrorMessage } from './ErrorMessage'
```

 **PROMPT DE IA** — Para mejorar cualquier componente visual:

Tengo este componente React de HookSuite: [pega el componente]. Quiero mejorar la UX añadiendo [describe qué quieres: animaciones de carga / transiciones suaves / feedback visual al copiar]. Usa solo Tailwind CSS y mantén el estilo actual del componente.

Paso 22 — Preparar el build de producción

22.1 Verifica que el proyecto construye sin errores:

```
npm run build
```

Si hay errores, léelos con atención. La mayoría son imports rotos o componentes que no exportan correctamente.

22.2 Previsualizas el build de producción localmente:

```
npm run preview
```

22.3 Cuando el build funcione limpio, haz commit:

```
git add .
git commit -m "feat(frontend): UX final pulida, estados vacíos y manejo de errores"
git push origin feature/frontend
```

Paso 23 — Abrir el Pull Request a main

23.1 Ve al repositorio en GitHub.

23.2 Haz clic en "Compare & pull request" para la rama `feature/frontend`.

23.3 Título del PR: `feat(frontend): módulo frontend completo HookSuite`

23.4 En la descripción del PR incluye:

- Lista de módulos implementados
- Capturas de pantalla de cada módulo funcionando
- Nota sobre qué funciona con mocks y qué necesita backend

23.5 Avisa a P5 GitHub para que revise y apruebe el PR.

📞 **CONTACTO CON P5 GitHub** — Abre el pull request en GitHub de la rama `feature/frontend` a `main` y notifica a P5 que está listo para revisión. P5 hará la revisión y el merge.

BLOQUE 08 — Documentación del módulo

Paso 24 — Capturas para el informe PDF

A medida que construyas cada módulo, toma capturas de pantalla con datos reales. Guárdalas en la carpeta `/docs/capturas/frontend/` del repositorio con nombres descriptivos:

- `proxy_panel_peticiones.png` — Panel de peticiones interceptadas con tráfico real
- `proxy_onboarding_pac.png` — Modal de configuración del PAC
- `repeater_peticion_respuesta.png` — Editor con una petición enviada y su respuesta
- `intruder_ataque_corriendo.png` — Tabla de resultados durante un ataque
- `vulnerabilidades_alerta_critica.png` — Panel de vulnerabilidades con detección real
- `utilidades_encoder.png` — Encoder en uso
- `red_tiempo_real.png` — Panel de red con paquetes marcados

🤖 **PROMPT DE IA** — Para escribir la descripción de cada captura en el informe:

Estoy redactando el informe técnico de HookSuite, una herramienta de pentesting web. Necesito escribir un pie de foto profesional para una captura que muestra [describe lo que muestra la captura]. El pie de foto debe ser conciso, técnico y en español. Dame tres opciones.

Paso 25 — Manual de uso del frontend

Escribe el manual de uso en el archivo `/docs/manual_frontend.md` del repositorio. Estructura mínima:

```
# Manual de uso – HookSuite Frontend

## 1. Acceso a la herramienta
Navega a [URL del servidor Hetzner]. La primera vez aparecerá el asistente de
configuración del proxy.

## 2. Configuración del proxy (primera vez)
[Describe el proceso con capturas]

## 3. Interceptar peticiones
[Describe cómo usar el panel Proxy]

## 4. Usar el Repeater
[Describe cómo modificar y reenviar peticiones]

## 5. Lanzar un ataque con el Intruder
[Describe el proceso de configuración y ejecución]

## 6. Interpretar las vulnerabilidades detectadas
[Describe el panel de vulnerabilidades]

## 7. Utilidades disponibles
[Describe cada utilidad]
```

 **PROMPT DE IA** — Para redactar cualquier sección del manual:

Estoy escribiendo el manual de usuario de HookSuite, una herramienta de pentesting web. Necesito redactar la sección de [nombre de la sección] explicando [describe qué hace esa sección]. El tono debe ser técnico pero claro, en español, dirigido a estudiantes de ciberseguridad. Dame la sección completa en formato Markdown.

Paso 26 — Script de demo para la presentación

Prepara el flujo exacto de la demo de 10 minutos. Escríbelo en `/docs/script_demo.md`:

```
# Script de demo – HookSuite (10 minutos)

## Minutos 1-2: Acceso y configuración del proxy
1. Abrir el navegador y navegar a [URL de Hetzner]
2. El modal de onboarding aparece automáticamente
3. Mostrar la detección automática del navegador y SO
```

4. Copiar la URL del PAC con el botón
5. Configurar el proxy en Chrome siguiendo las instrucciones
6. El modal se cierra automáticamente al detectar el proxy activo

Minutos 2-4: Interceptar tráfico real

1. Navegar a la web de prueba con el proxy activo
2. Mostrar cómo las peticiones aparecen en tiempo real en el panel Proxy
3. Hacer click en una petición interesante para ver el detalle
4. Mostrar headers, body y respuesta

Minutos 4-6: Repeater

1. Hacer clic en "Enviar al Repeater" en una petición interceptada
2. Modificar un parámetro en el editor
3. Enviar la petición modificada
4. Mostrar la respuesta y compararla con la original

Minutos 6-8: Intruder automatizado

1. Ir al módulo Intruder
2. Configurar la URL objetivo y el punto de inyección
3. Seleccionar tipo SQL Injection
4. Iniciar el ataque y mostrar la tabla de resultados en tiempo real
5. Señalar las filas marcadas como vulnerables por la IA

Minutos 8-10: Vulnerabilidades detectadas

1. Ir al panel de Vulnerabilidades
2. Mostrar la alerta crítica de SQL Injection
3. Expandir el detalle: payload, evidencia, recomendación de remediación
4. Ir al panel de Red y mostrar el tráfico con paquetes marcados

Ensayar este script al menos dos veces antes del día 25. Graba una sesión de práctica como backup.

Checklist final antes de la entrega

Antes del día 22 (tres días antes de la entrega), verifica que todo está en orden:

- ☐ El proyecto hace build sin errores con `npm run build`
- ☐ Todos los módulos son accesibles desde el sidebar
- ☐ El panel Proxy muestra peticiones (con mocks o datos reales)
- ☐ El Repeater envía peticiones y muestra respuestas
- ☐ El Intruder lanza ataques y muestra resultados
- ☐ Las utilidades (encoder, hash, regex, payloads) funcionan
- ☐ El panel de vulnerabilidades muestra alertas
- ☐ El panel de red muestra paquetes con marcado de color
- ☐ El onboarding del PAC funciona y detecta el navegador/SO
- ☐ El importador de peticiones (raw HTTP y cURL) funciona
- ☐ Todos los commits en la rama siguen la convención
- ☐ El PR a main está aprobado por P5
- ☐ Las capturas para el informe están en `/docs/capturas/frontend/`
- ☐ El manual de uso está escrito en `/docs/manual_frontend.md`

- ☐ El script de demo está en `/docs/script_demo.md`
 - ☐ La demo ha sido ensayada mínimo dos veces
 - ☐ Hay una grabación de backup de la demo funcionando
-

Manual técnico P1 Frontend — Proyecto HookSuite — Módulo de Ciberseguridad Avanzada 2026